



FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGIES

THE ICT UNIVERSITY – SUMMER 2026

Francanglais Studio

A Lexical and Syntactic Analyzer for the
Informal Urban Speech of Yaoundé

Software Design & Architecture Recommendation Document

CS4110 – Compiler Construction • Credit Value 6

Instructor: Engr. Tanwi Nkiamboh

Prepared by the project team

Kamdeu Yamdjeuson Neil Marshall	ICTU20241386
Eyong Seanna Tabe	ICTU20241297
Tuheu Tchoubi Pempeme Moussa Fahdil	ICTU20241393

This document explains the design, the compiler theory applied, and the full source-code architecture of the **Language-Analyser** project: a corpus-driven lexer, an LL(1) grammar-preparation and parsing pipeline, a FastAPI backend, and a React frontend, deployed as a single containerised service.

September 15, 2026

Contents

1	Project Context and Objective	2
1.1	The assignment	2
1.2	Our interpretation	2
2	System Architecture	2
2.1	High-level view	2
2.2	Repository layout	3
3	The Compiler Pipeline	3
3.1	Overview	3
3.2	Stage 1 – Span scanning (<code>core/text.py</code>)	3
3.3	Stage 2 – Lexical analysis (<code>core/lexer.py</code> , <code>core/lexicon.py</code>)	4
3.4	Stage 3 – Grammar preparation (<code>core/grammar.py</code>)	4
3.5	Stage 4 – FIRST/FOLLOW/table construction (<code>core/ll1.py</code>)	5
3.6	Stage 5 – Table-driven LL(1) parsing (<code>core/parser.py</code>)	5
3.7	Worked example on the demonstration grammar	6
3.8	Illustrated parse trace	6
4	Core Data Contracts (<code>core/models.py</code>)	7
5	UML Diagrams	8
5.1	Class diagram – grammar and parsing model	8
5.2	Class diagram – corpus and analysis model	9
5.3	Sequence diagram – “Analyze this statement” request	9
5.4	Use-case diagram	10
6	Corpus, Snapshots and Statistics	11
6.1	Provenance-aware corpus model (<code>core/corpus.py</code>)	11
6.2	Statistics (<code>core/analysis.py</code>)	12
7	API Layer (FastAPI)	12
7.1	Application factory	12
7.2	Routes	12
7.3	Security posture	12
8	Persistence (<code>storage/</code>)	12
9	Frontend (React + TypeScript + Vite)	13
10	Deployment Recommendation	13
11	Verification Strategy	14
12	Scope Boundaries	14
13	Conclusion	14

1 Project Context and Objective

1.1 The assignment

The CS4110 *Compiler Construction* final examination asks each group to build a mini-language analyzer over **real, manually transcribed** statements collected in Yaoundé – a city where everyday speech mixes French, English, Pidgin, Fulfulde, Ewondo expressions and city slang (“*franc-anglais*”). The graded deliverable requires: a corpus of 10–15 authentic statements; a lexical specification with regular-expression token types; a hand-derived context-free grammar (CFG) reduced to LL(1) form (left-recursion removal, left factoring, FIRST/FOLLOW sets, parsing table); a working parser that accepts or rejects the collected sentences; and a written report of 25–30 pages with source code and a live demonstration.

1.2 Our interpretation

Rather than a one-off script, the team designed **Francanglais Studio**: a small but complete compiler front end (lexer, grammar preparation, LL(1) parser) wrapped in a web application so that the analyzer, the corpus, the grammar derivation, and the statistics can all be inspected and demonstrated interactively, while keeping every compiler-theory artefact – tokens, FIRST/FOLLOW sets, the parsing table, the parse trace – fully visible and reproducible, exactly as the grading rubric asks.

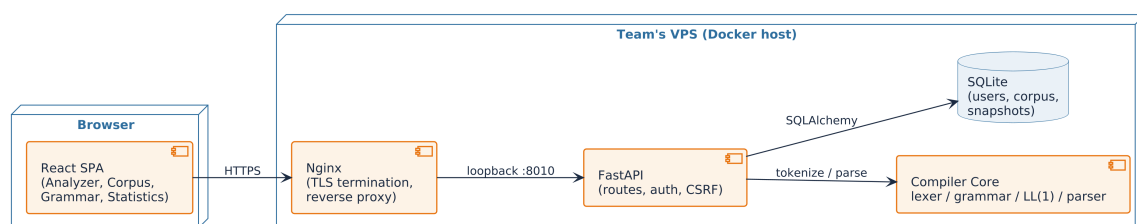
Guiding principle

The analyzer must recognise a *documented subset* of collected speech. It never pretends to understand every possible sentence, and it never silently drops an unknown word or silently resolves a grammar ambiguity – both are reported explicitly.

2 System Architecture

2.1 High-level view

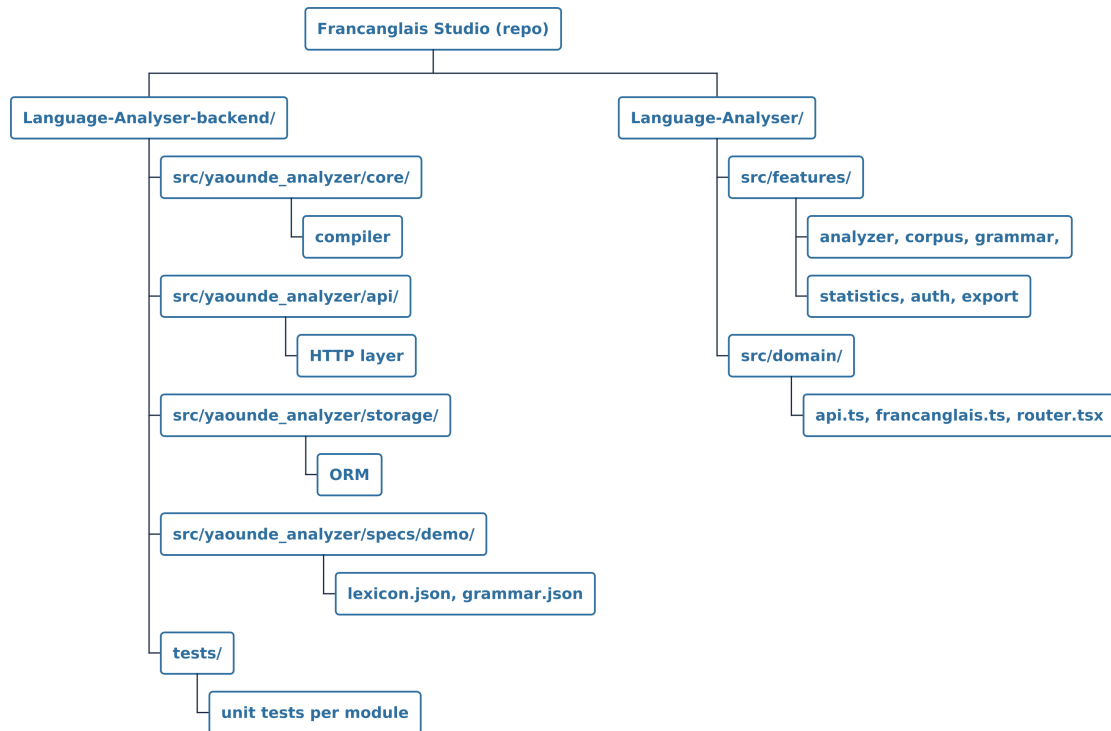
The system is split into three tiers that mirror the classic compiler-course separation between the *analyzer core*, the *service that exposes it*, and the *client that visualises it*.



- **Compiler core** (`yaounde_analyzer/core/`) – pure Python, no web or database dependency. This is the graded compiler: `text.py`, `lexicon.py`, `lexer.py`, `grammar.py`, `ll1.py`, `parser.py`, `analysis.py`, `models.py`, `corpus.py`.
- **API layer** (`yaounde_analyzer/api/`) – a FastAPI application that loads the compiler core once at startup, validates that the grammar is genuine LL(1), and exposes it over HTTP with authentication, corpus editing and export.
- **Frontend** (`Language-Analyser/`) – a React + TypeScript + Vite single-page app with four work views that mirror the four graded components of the assignment: *Analyzer*, *Corpus*, *Grammar*, *Statistics*.

- **Deployment** – one Docker image containing the built frontend and the FastAPI app, bound to 127.0.0.1:8010 behind the team’s existing Nginx host, so the demonstration works both online and offline from the presentation laptop.

2.2 Repository layout



3 The Compiler Pipeline

3.1 Overview

Every statement – whether typed for a live demo or replayed from the stored corpus – travels through the same five stages. This is the direct software realisation of the lexical/syntactic pipeline required by the assignment.



The grammar-preparation stage (`grammar.py` + `ll1.py`) is **not** repeated per statement: `PreparedAnalyzer` (in `analysis.py`) builds the lexicon, transforms the grammar, and computes FIRST/FOLLOW/table exactly once when the application starts, then reuses that prepared table for every request – this is the standard compiler-construction separation between *compile-time grammar analysis* and *run-time parsing*.

3.2 Stage 1 – Span scanning (`core/text.py`)

A first regular-expression pass turns the raw Unicode string into a flat, gapless sequence of spans, each tagged `WORD`, `NUMBER`, `SPACE`, `NEWLINE`, `PUNCTUATION` or `OTHER`. Every code point of the input is accounted for by exactly one span – nothing is skipped silently at this stage; word forms are also *canonicalised* (case, accents) so that the lexicon lookup that follows is consistent.

3.3 Stage 2 – Lexical analysis (core/lexer.py, core/lexicon.py)

This is the “regular expressions for token types” and “custom lexical analyzer” required by the specification. Instead of a generic dictionary, the lexicon (specs/demo/lexicon.json) is a **corpus-driven** list of `LexicalEntry` records – each one only exists because it was attested in a reviewed Yaoundé statement, with its part of speech, slang flag and language candidate (French / English / Pidgin / Fulfulde / Ewondo / mixed).

Listing 1: Longest-match tokenisation strategy (simplified from `lexer.py`)

```

1 for each WORD span in order:
2     canonical = canonicalize_word(span.raw)
3     try every multiword lexicon entry starting with ‘canonical’,
4         longest phrase first (e.g. "hala me small money" before "hala
           me")
5     if none matches:
6         try the single-word lexicon entry for ‘canonical’
7     if still nothing:
8         emit an UNKNOWN token    # never dropped, never guessed

```

Unknown vocabulary is a *first-class, visible* outcome (terminal UNKNOWN), which is exactly what lets the report discuss “why Yaoundé communication is linguistically complex” with concrete evidence instead of hiding gaps.

Design recommendation applied

Keep raw wording, canonical form, grammatical category, language label and slang status as *separate* fields on `Token` (see §4). This lets the Statistics view report raw-frequency, canonical-frequency and terminal- frequency independently, satisfying the “analyze token frequency and variation” requirement without re-deriving anything downstream.

3.4 Stage 3 – Grammar preparation (core/grammar.py)

The team’s context-free grammar is authored in its natural, descriptive form (the JSON file `specs/demo/grammar.json`) and then mechanically reduced to an LL(1)-ready grammar, with every transformation kept in a visible *ledger* (`TransformationStep`) so the report can show “before” and “after” productions side by side, as the rubric requires.

Nullable set. Computed to a fixed point: a nonterminal is nullable if any of its right-hand sides is empty or made entirely of nullable symbols.

Left-recursion detection and elimination. The module builds a *left-corner graph*: an edge $X \rightarrow Y$ exists whenever some production $X \rightarrow \alpha Y \dots$ has a nullable prefix α . A self-loop (A) is direct left recursion; a longer cycle is indirect left recursion, which is *detected but not silently rewritten* – the team judged automatic indirect elimination out of scope and, if it occurs, corrects the grammar by hand instead. Direct recursion

$$A \rightarrow A\alpha_1 \mid \dots \mid A\alpha_n \mid \beta_1 \mid \dots \mid \beta_m$$

is rewritten as

$$A \rightarrow \beta_1 A' \mid \dots \mid \beta_m A' \quad A' \rightarrow \alpha_1 A' \mid \dots \mid \alpha_n A' \mid \varepsilon$$

using a freshly minted nonterminal A' (suffixed with a prime, disambiguated if already taken).

Left factoring. Alternatives of the same nonterminal are grouped by their first RHS symbol; a group of two or more sharing a symbol is factored on their *longest common prefix*, run to a fixed point since a factored remainder can itself expose a further common prefix.

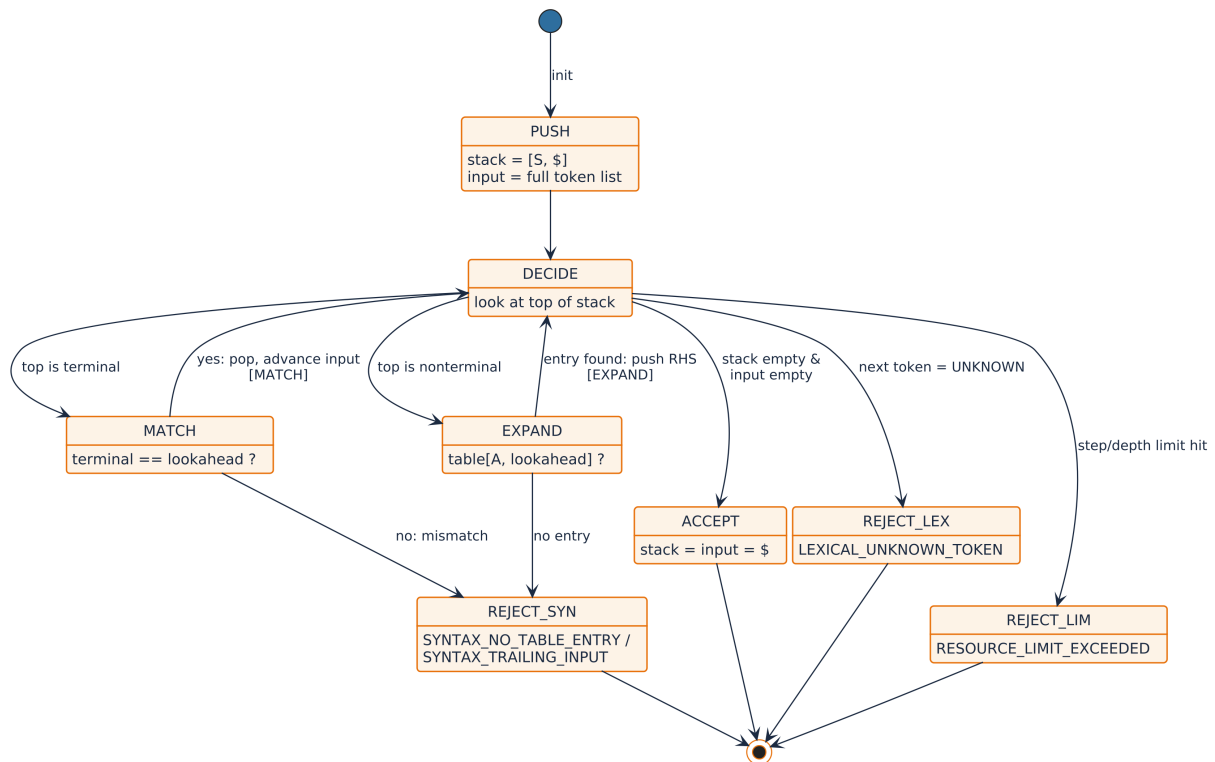
3.5 Stage 4 – FIRST/FOLLOW/table construction (core/ll1.py)

Working on the already-transformed grammar:

- $\text{FIRST}(t) = \{t\}$ for every terminal; $\text{FIRST}(A)$ for a nonterminal is the fixed-point union, over each production $A \rightarrow X_1X_2\dots X_k$, of $\text{FIRST}(X_1)$, then $\text{FIRST}(X_2)$ if X_1 is nullable, and so on.
- $\text{FOLLOW}(S)$ is seeded with the reserved end-of-input symbol $\$$; for every occurrence of a nonterminal inside a production's right-hand side, FIRST of what follows it is added to its FOLLOW set, and if that remainder is entirely nullable, FOLLOW of the producing nonterminal is added as well.
- The LL(1) table's **SELECT** set for a production $A \rightarrow \alpha$ is $\text{FIRST}(\alpha)$, plus $\text{FOLLOW}(A)$ when α is nullable. A table cell claimed by more than one production is a genuine **FIRST/-FIRST or FIRST/FOLLOW conflict**: it is *reported*, not arbitrarily resolved by “picking the first rule”, and the application refuses to start serving traffic if any conflict remains (`PreparedAnalyzer.check()` raises `ParserConfigurationError` at boot).

3.6 Stage 5 – Table-driven LL(1) parsing (core/parser.py)

The parser is a classic **explicit-stack pushdown automaton**, not a recursive descent implementation – this was a deliberate choice so that every stack configuration can be recorded as a **TraceStep** and later replayed in the frontend's synchronised trace/parse-tree view. The automaton below is the formal state machine that `parser.py` implements: five working states (**PUSH**, **DECIDE**, **MATCH**, **EXPAND**) and four terminal states (**ACCEPT** and three distinct **REJECT_*** states, one per failure reason).



- **PUSH** – initial configuration: $\text{stack} = [\$, S]$, $\text{input} = \text{full token terminal sequence}$.
- **MATCH** – top of stack is a terminal equal to the lookahead: pop, and advance the input; the corresponding parse-tree leaf records its token span.

- **EXPAND** – top of stack is a nonterminal A : consult `table[A,lookahead]`; push the chosen production’s RHS in reverse; record the production id and grow a parse-tree node with one child per RHS symbol.
- **ACCEPT** – stack and input both reduce to $\$$ together.
- **REJECT** – distinguished by *reason*, not just “failed”: `LEXICAL_UNKNOWN_TOKEN`, `SYNTAX_NO_TABLE_ENTRY`, `SYNTAX_TRAILING_INPUT`, or `RESOURCE_LIMIT_EXCEEDED` (a bounded step/stack-depth guard so a pathological input can never hang the analyzer).

Every step – `stack`, `remaining_terminals`, `action`, and (when relevant) `production_id` – is appended to an immutable `TraceStep` tuple. `ParseResult` additionally records `matched_token_count`, `failing_position`, and `expected_terminals`, giving exactly the “accepted / rejected” evidence and diagnostics required for the report and demonstration.

3.7 Worked example on the demonstration grammar

The bundled demo grammar (`specs/demo/grammar.json`) is a small five-terminal Subject–Predicate grammar used to validate the pipeline end to end before the real corpus replaces it:

$$\begin{aligned}
 S &\rightarrow NP VP \\
 NP &\rightarrow DET NOUN \mid PRON \mid NOUN \\
 VP &\rightarrow VERB \mid VERB NP \mid VERB PP \\
 PP &\rightarrow PREP NP
 \end{aligned}$$

This grammar contains no left recursion and no shared alternatives, so Stage 3 (§3.4) leaves it unchanged; Stage 4 (§3.5) produces the following FIRST/FOLLOW sets and conflict-free LL(1) table:

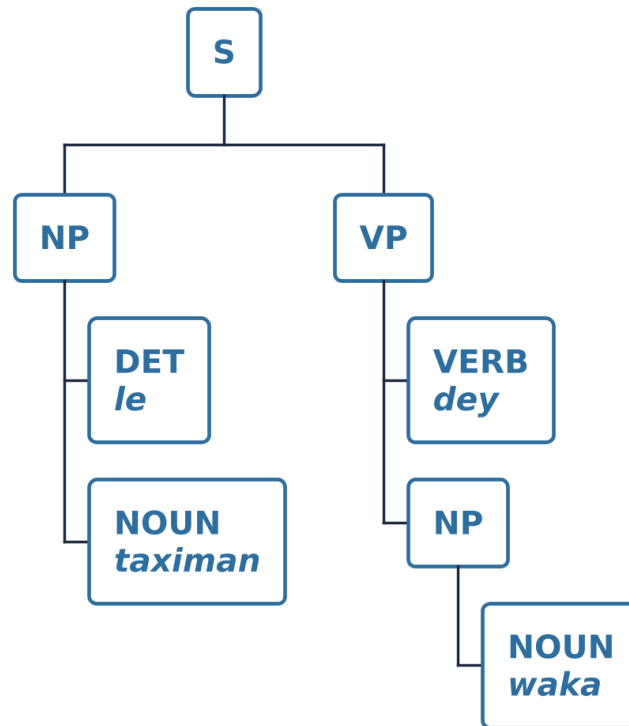
Nonterminal	FIRST	FOLLOW
S	DET, PRON, NOUN	$\$$
NP	DET, PRON, NOUN	VERB, $\$$, PREP
VP	VERB	$\$$
PP	PREP	$\$$

	DET	NOUN	PRON	VERB	PREP
S	$NPVP$	$NPVP$	$NPVP$	–	–
NP	$DET NOUN$	$NOUN$	$PRON$	–	–
VP	–	–	–	$VERB \mid VERB NP \mid VERB PP^*$	–
PP	–	–	–	–	$PREP NP$

* The three VP alternatives share terminal $VERB$ in their *FIRST* sets; because the demo grammar has not yet been left-factored on VP , `build_ll1_table` reports this as a genuine *FIRST/FIRST* conflict rather than guessing – illustrating why Stage 3’s left-factoring pass exists, and why the real corpus grammar must be re-run through it once collected.

3.8 Illustrated parse trace

For the accepted sentence “*le taximan dey waka*” (tokens $DET NOUN VERB VERB$ under the corpus lexicon), the automaton produces the following stack evolution and parse tree:



Step	Action	Stack (top → right)	Lookahead
1	PUSH	<i>S</i> , \$	DET
2	EXPAND (<i>S</i> → <i>NP VP</i>)	<i>NP</i> , <i>VP</i> , \$	DET
3	EXPAND (<i>NP</i> → <i>DET NOUN</i>)	<i>DET</i> , <i>NOUN</i> , <i>VP</i> , \$	DET
4	MATCH	<i>NOUN</i> , <i>VP</i> , \$	NOUN
5	MATCH	<i>VP</i> , \$	VERB
6	EXPAND (<i>VP</i> → <i>VERB NP</i>)	<i>VERB</i> , <i>NP</i> , \$	VERB
7	MATCH	<i>NP</i> , \$	VERB
8	EXPAND (<i>NP</i> → <i>NOUN</i>)	<i>NOUN</i> , \$	VERB
9	MATCH	\$	\$
10	ACCEPT	–	–

For a sentence with a token outside the corpus lexicon (e.g. an unattested slang word), the parser stops at Stage 2 already: `parse_tokens` returns `accepted=False`, `rejection_reason=LEXICAL_UNKNOWN_TOKEN` and the exact character offset – the “show which sentences are accepted or rejected” requirement is satisfied with a precise, reproducible reason rather than a bare boolean.

4 Core Data Contracts (`core/models.py`)

All compiler stages communicate through frozen Pydantic models, so every artefact the report needs to show is already a serialisable, validated object rather than an ad-hoc dictionary.

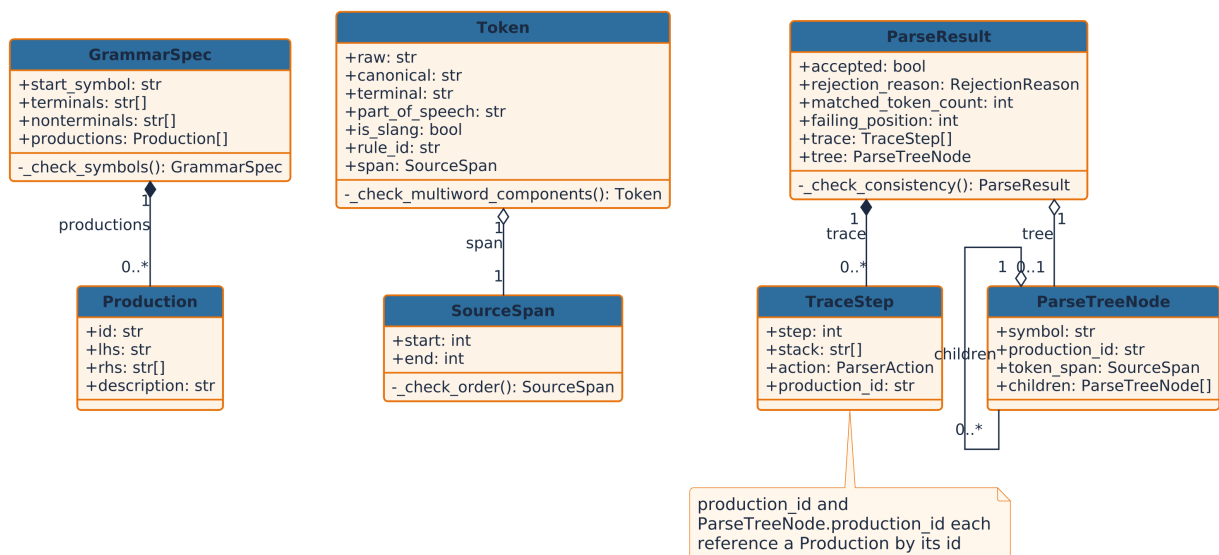
Model	Role
SourceSpan	Half-open $[start, end)$ Unicode offset into the raw statement; attached to every token and parse-tree leaf so the frontend can highlight exact substrings.
Token	<code>raw</code> , <code>canonical</code> , <code>terminal</code> , <code>part_of_speech</code> , <code>language_candidates</code> , <code>is_slang</code> , <code>is_multiword</code> , <code>component_spans</code> , <code>rule_id</code> , <code>span</code> .
GrammarSpec	<code>start_symbol</code> , disjoint <code>terminals/nonterminals</code> , and a validated <code>Production</code> tuple (rejects undeclared symbols, duplicate ids, and re-use of the reserved \$).
TransformationStep	One ledger entry: kind (left-recursion elimination / left factoring), human description, and the exact source/result production ids – the “show steps” evidence.
TraceStep, ParseTreeNode, ParseResult	The automaton’s full execution trace, resulting parse tree, and accept/reject verdict with reason (§3.6).
TopicMatch	A transparent topic label plus the exact canonical words that triggered it – never a black-box classification.
AnalysisResult	The per-statement bundle returned to callers: tokens + parse + topics, keyed to an exact statement revision.

Validators enforce compiler-theory invariants directly in the data layer – e.g. a `GrammarSpec` cannot declare the same identifier as both a terminal and a nonterminal, and a `ParseResult` cannot be simultaneously accepted and carry a rejection reason.

5 UML Diagrams

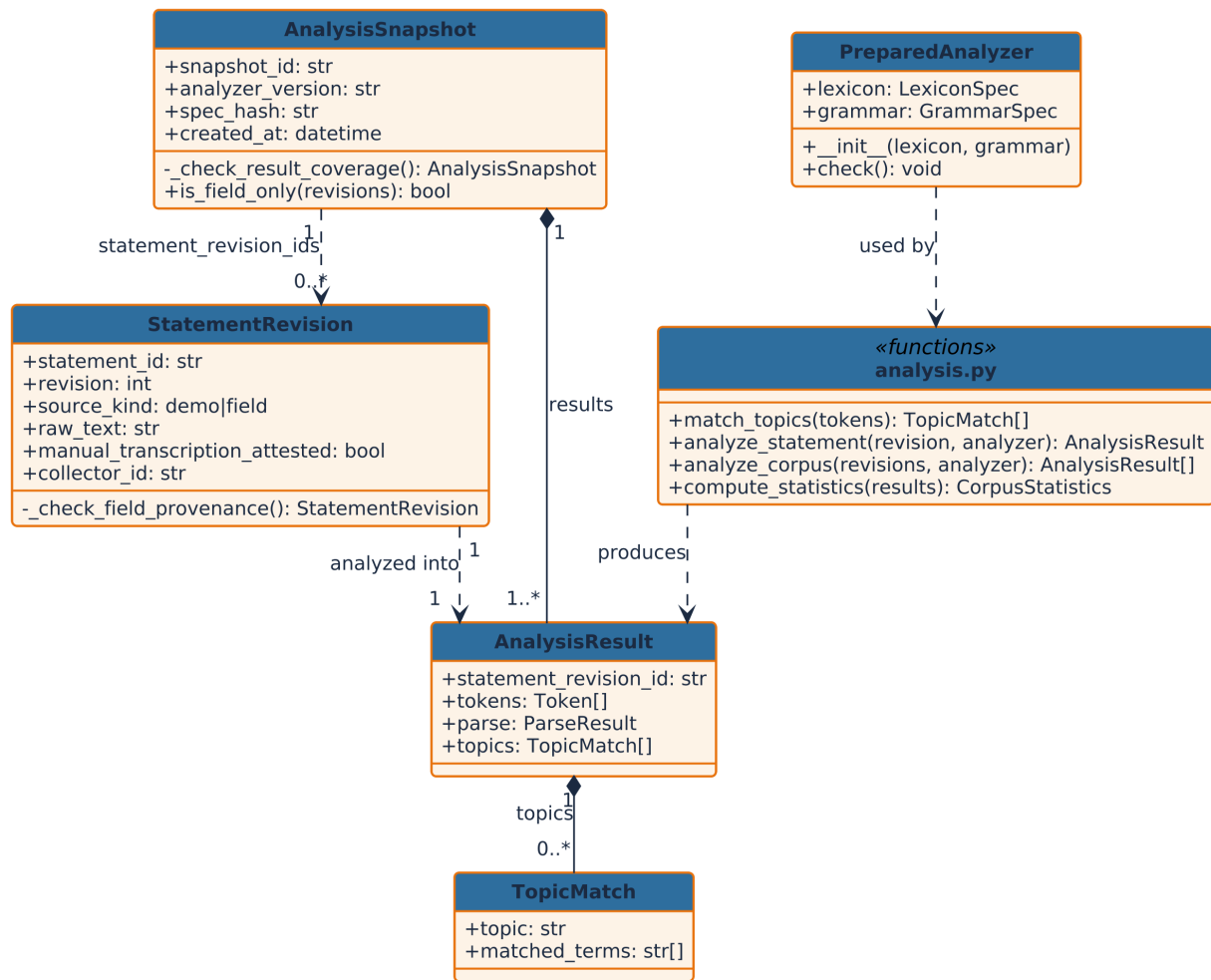
The class relationships, run-time message flow and system actors described in words above are restated here as standard UML diagrams, so the report can point to a single authoritative picture for each.

5.1 Class diagram – grammar and parsing model



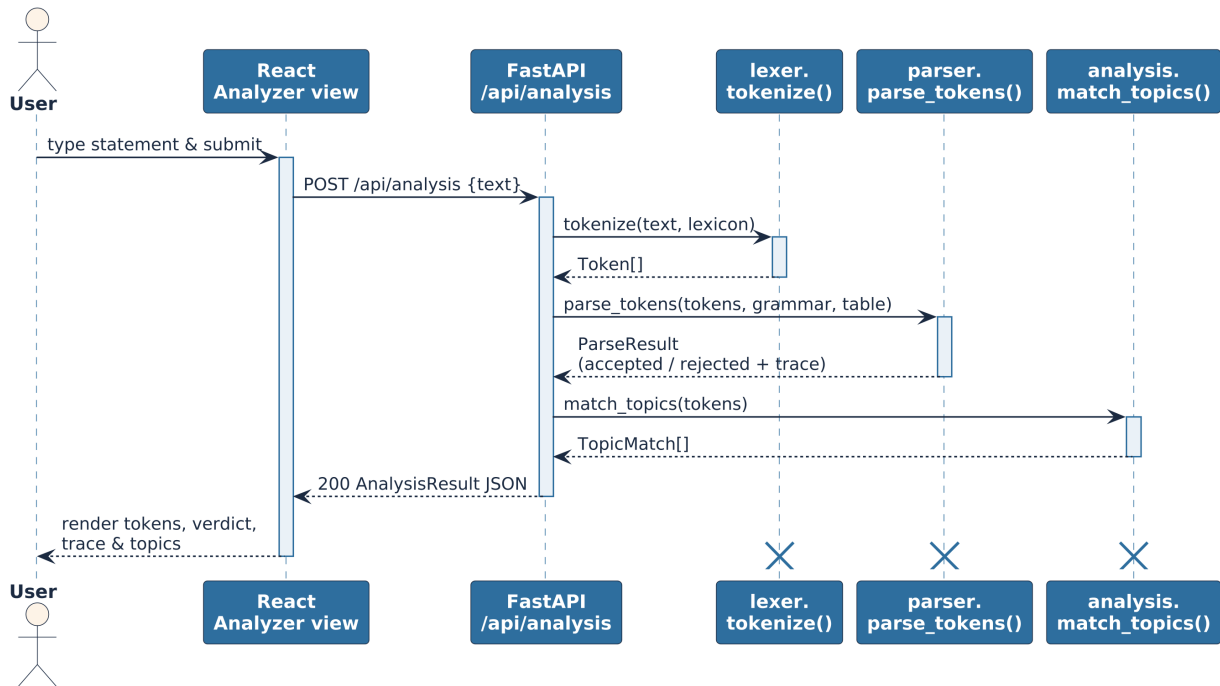
Filled diamond = composition; hollow diamond = aggregation; open arrow = association. Leading - marks a private/internal operation (a Pydantic model validator).

5.2 Class diagram – corpus and analysis model



5.3 Sequence diagram – “Analyze this statement” request

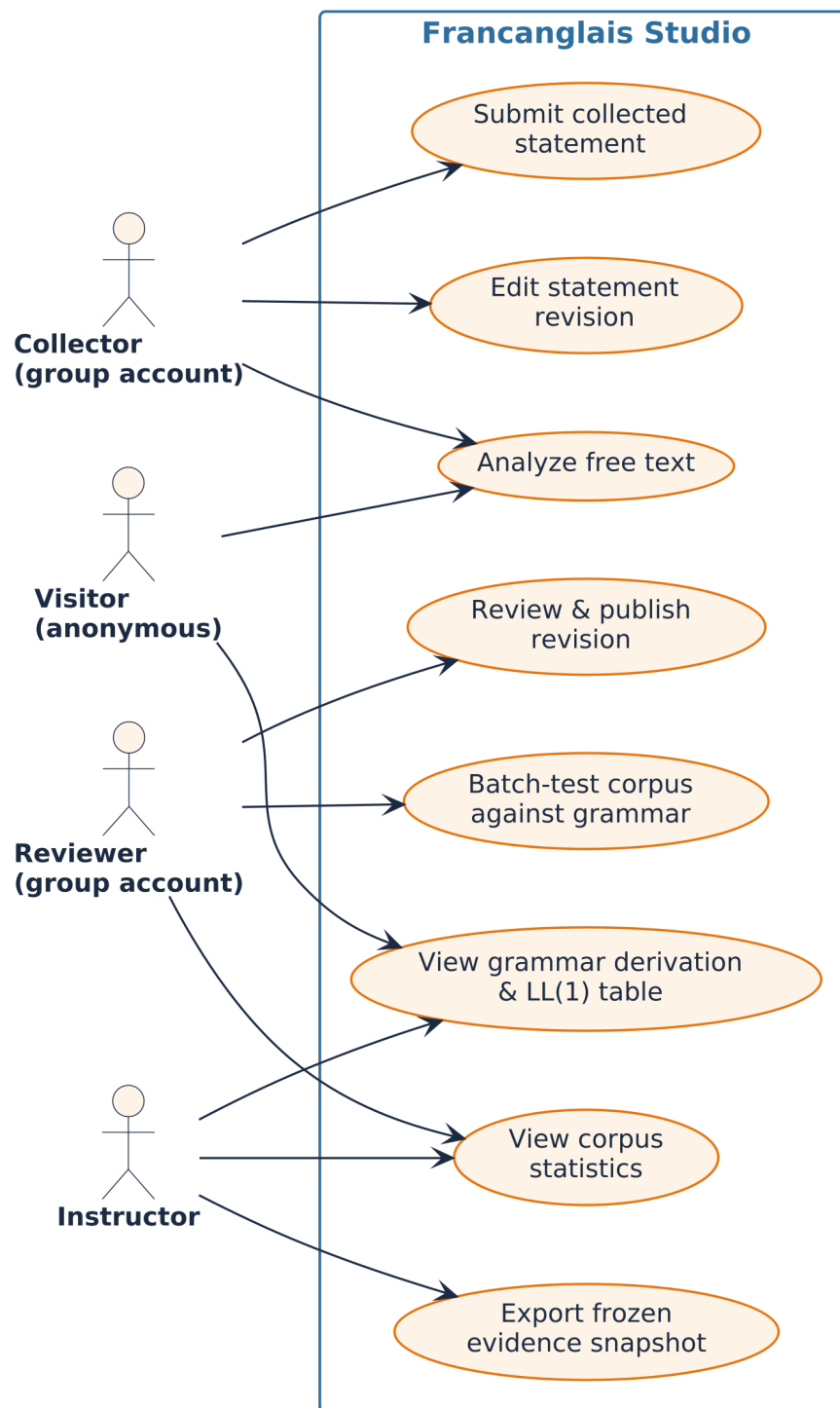
The most-used interaction: a statement typed in the Analyzer view (or replayed from the corpus) is tokenized, parsed and topic-matched in one request, reusing the already-prepared grammar. Activation bars show exactly how long each participant is busy; the X markers at the foot of the three helper life-lines are UML *destroy* markers – those call-scoped collaborators do not persist between requests, unlike **PreparedAnalyzer** which lives for the whole process.



The grammar, lexicon and $LL(1)$ table used by steps 2 and 4 are not rebuilt per request – they come from the single *PreparedAnalyzer* instance built once at application startup (§3.4–§3.5).

5.4 Use-case diagram

Generated with PlantUML for cleaner routing between actors and use cases than a hand-placed diagram allows.



6 Corpus, Snapshots and Statistics

6.1 Provenance-aware corpus model (core/corpus.py)

A `StatementRevision` is immutable and append-only: editing a statement creates a new revision rather than overwriting history. Field-sourced statements (`source_kind="field"`) must carry `manual_transcription_attested`, enforced by a validator – so a statement cannot enter the graded corpus without an explicit human attestation that it was listened to and transcribed by hand, as the assignment demands. Demonstration data (`source_kind="demo"`) is kept distinct and `AnalysisSnapshot.is_field_only()` lets the export pipeline verify that a submitted snapshot contains no synthetic examples.

6.2 Statistics (`core/analysis.py`)

`compute_statistics` folds a batch of `AnalysisResults` into raw, canonical and terminal frequency counters plus the distinct set of unknown words – directly answering “analyze token frequency and variation”. Topic matching (`match_topics`) uses a small, evidence-cited keyword table (`TOPIC_KEYWORDS`) covering the ten required themes (commuting, internet, electricity, market bargaining, rain, fuel, roadside business, bendskin, security, university life); a topic is reported together with the exact words that triggered it and *never* influences grammatical acceptance.

7 API Layer (FastAPI)

7.1 Application factory

`api/main.py` builds the app with no import-time side effects: the demo lexicon and grammar are loaded and validated with `PreparedAnalyzer.check()` *before* the server accepts traffic, so an invalid (non-LL(1)) grammar fails fast at boot instead of surfacing as a runtime 500 error later. Health endpoints (`/api/health/live`, `/api/health/ready`) let the deployment distinguish “process is up” from “the parsing table is actually conflict-free”.

7.2 Routes

Router	Responsibility
<code>auth</code>	Cookie-based session login/logout for the three provisioned group accounts; issues a CSRF token alongside the session.
<code>analysis</code>	Anonymous, non-persistent tokenize-and-parse for the live Analyzer view; returns the full <code>AnalysisResult</code> .
<code>corpus</code>	Protected create/edit/review/publish of statement revisions, with revision checks so two collaborators cannot silently overwrite each other.
<code>grammar</code>	Serves the regex/lexicon catalogue, the transformation ledger, FIRST/FOLLOW sets and the LL(1) table for the Grammar view.
<code>statistics</code>	Corpus-wide frequency, variation and topic results for the Statistics view.
<code>export</code>	Reproducible, version-tied snapshot export (source data + grammar + lexer versions) for the written report.

7.3 Security posture

Sessions store only the SHA-256 hash of the bearer token (`AuthSession`); CORS is restricted to configured origins; state-changing routes require a matching `x-csrf-token` header. This satisfies the plan’s verification checklist (authentication, logout, CSRF, simultaneous-edit protection, public/private separation) without adding unneeded complexity such as unrestricted grammar editing or multi-tenant accounts, which were deliberately kept out of v1.

8 Persistence (`storage/`)

SQLAlchemy ORM rows are kept deliberately separate from the Pydantic contracts in `core/`: `core` models are what the compiler reasons about, `storage` rows are what gets persisted and are converted at the API boundary. Key tables: `users`, `auth_sessions`, `statement_records` / `statement_revisions` (append-only, with a nullable “currently published” pointer), and `analysis_snapshots` (one persisted result per exact revision). SQLite is used with a persistent Docker volume and scheduled backups, appropriate for a small graded-project corpus of a few dozen statements.

9 Frontend (React + TypeScript + Vite)

The frontend is organised feature-first under `src/features/`, one folder per work view, plus a shared `src/domain/` for the typed API client (`api.ts`), shared Francanglais types (`francanglais.ts`) and routing (`router.tsx`).

View	What it shows
analyzer	Free-text input, highlighted token spans, accept/reject verdict, and the synchronised stack-trace / parse-tree visualiser (§3.6).
corpus	Statement collection, editing, review workflow and batch testing against the current grammar.
grammar	The regex/lexicon catalogue, the before/after transformation ledger, FIRST/FOLLOW tables and the LL(1) parsing table.
statistics	Frequency and variation charts, topic-match results, and reproducible export links.
auth, export	Session login for the three group accounts, and download of a frozen, versioned evidence snapshot.

All four work views open directly on the Analyzer by default, with responsive tables so the 10-minute live demonstration can be given from any laptop or projector.

10 Deployment Recommendation

Recommended topology

Build the Docker image *outside* the VPS (CI or local machine) and ship only the image. Bind the container's application port to `127.0.0.1:8010` – never expose it directly – and add a dedicated virtual host on the team's existing Nginx that terminates HTTPS and proxies to that loopback port, alongside (not replacing) any existing sites on the same server. Enable health checks, basic rate limiting, non-root container execution, a persistent volume for the SQLite file, and modest CPU/memory limits appropriate for a student-project workload.

Operational checklist before the demonstration:

1. Scheduled SQLite backups, with at least one *tested* restore into a separate volume.
2. Keep the previous container image tagged for instant rollback.
3. Mirror the exact same image locally on the presentation laptop, so the 10-minute demo does not depend on venue internet access.

11 Verification Strategy

Layer	Checks
Compiler core	Hand-checked token expectations; Unicode and multiword span boundaries; nullable-set correctness; FIRST/FIRST and FIRST/FOLLOW conflict detection; unknown, missing and trailing-token handling (see <code>tests/test_lexer.py</code> , <code>test_grammar.py</code> , <code>test_ll1.py</code> , <code>test_parser.py</code>).
Application	Login/logout, CSRF enforcement, concurrent-edit protection, public/private visibility, exclusion of demo data from graded results (<code>test_api.py</code> , <code>test_corpus.py</code>).
Browser	Cross-device checks on the four work views: analysis, trace navigation, corpus editing, export, readable layouts on desktop and mobile widths.
Deployment	HTTPS and deep links resolve; the application port is unreachable from outside loopback; data survives a container restart; backup restore succeeds; existing sites on the shared Nginx are unaffected.
Academic	10–15 reviewed authentic statements collected; complete lexical/syntactic evidence generated from one frozen snapshot; rendered report within 25–30 pages; an offline rehearsal of the 3-minute-per-student, 10-minute-total presentation.

12 Scope Boundaries

To keep the graded deliverable focused on the required compiler-construction evidence, the team deliberately left the following *outside* version 1: automatic transcription of audio, LLM-assisted parsing or grammar induction, unrestricted in-browser grammar editing, additional group accounts beyond the three provisioned collectors, and a second parsing algorithm (e.g. SLR(1)) alongside LL(1). None of these strengthen the specific LL(1) derivation the rubric asks for, and each would add surface area that competes with report-writing and rehearsal time in the two remaining weeks before submission.

13 Conclusion

Francanglais Studio turns the CS4110 specification into a working, inspectable compiler: a corpus-driven lexer that never silently discards a word, a grammar pipeline that documents every left-recursion-elimination and left-factoring step, an LL(1) table that refuses to hide a genuine ambiguity, and an explicit-stack parser whose entire execution trace can be replayed. Wrapping that compiler in a small FastAPI service and a four-view React frontend does not change what is being graded – it makes the same lexical/syntactic evidence the rubric asks for directly visible and demonstrable, both online and offline, in the 10-minute presentation window.